

Продвинутое машинное обучение

Лекция 2: CART: Classification and Regression Trees

Автор курса: Лапшина Анастасия (University of Maryland, College Park, ВШЭ)

Часть 1. Решающие деревья как класс функций

Решающие деревья

– это класс кусочно-постоянных функций, которые разбивают пространство признаков на попарно непересекающиеся области (листья) и в каждой области выдают константное предсказание:

$$\mathcal{X} = \bigcup_{m=1}^M R_m \quad \mathcal{X} \subseteq \mathbb{R}^d$$

$$R_m \cap R_{m'} = \emptyset \quad (m \neq m'), \quad \bigcup_{m=1}^M R_m = \mathcal{X}$$

$$\hat{f}(x) = \sum_{m=1}^M c_m \mathbb{1}\{x \in R_m\}$$

$$x \in R_m \implies \hat{f}(x) = c_m$$

Объект попадает в ту область (лист), которой принадлежит его вектор признаков x

Бинарное разбиение в вершине

$$R_L(j, s) = \{x \in R : x_j \leq s\}.$$

$$R_R(j, s) = \{x \in R : x_j > s\}.$$

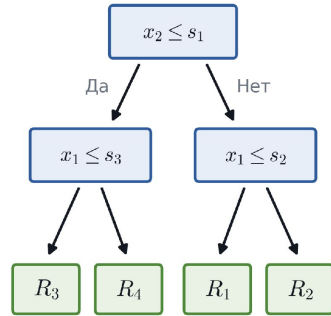
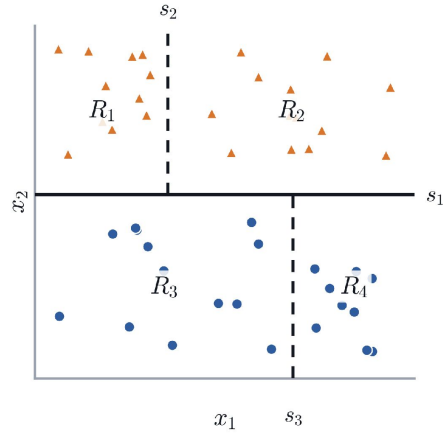
$$R = R_L(j, s) \sqcup R_R(j, s).$$

Каждая внутренняя вершина выбирает признак j и порог s , после чего текущая область R разбивается на две непересекающиеся подобласти

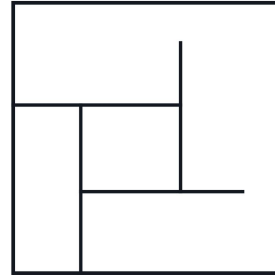
В классическом CART сплит axis-aligned: условие зависит только от **одного** признака.

Предикат имеет вид $x_j \leq s$

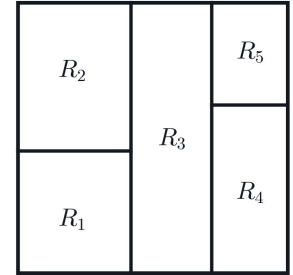
Разбиение пространства признаков



нельзя получить
деревом



получается
вложенными сплитами



Часть 2. Классификация и Регрессия

Задача классификации

Пусть N_m - число обучающих примеров, попавших в R_m .
Тогда оценка вероятности класса k в листе m :

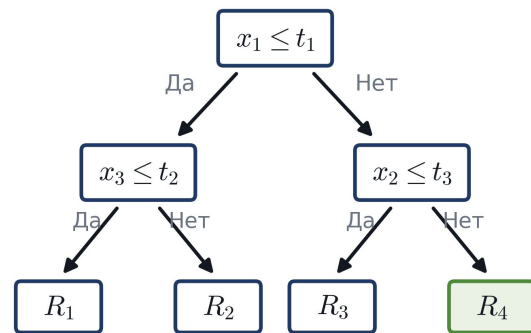
$$\hat{p}_{mk} = \frac{1}{N_m} \sum_{x_i \in R_m} I(y_i = k), \quad k = 1, \dots, K$$

Тогда предсказываем мажоритарный класс в листе m :

$$k(m) = \arg \max_k \hat{p}_{mk}$$

Функция классификации дерева:

$$x \in R_m \Rightarrow \hat{f}(x) = k(m)$$



Лист R_4			
класс	N_{4k}	$\hat{p}_{4k}$	
1	2	0.250	
2	5	0.625	
3	1	0.125	
$N_4 = 8 \rightarrow k(4) = \arg \max_k \hat{p}_{4k} = 2$			

Критерии CART для классификации

Идея: выбираем split, который делает дочерние узлы более однородными

Для этого для каждого разбиения считаем взвешенную нечистоту дочерних узлов

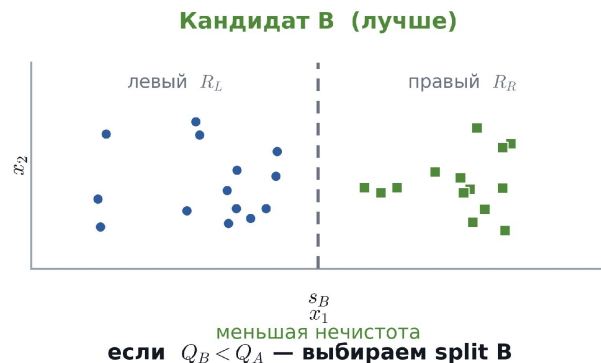
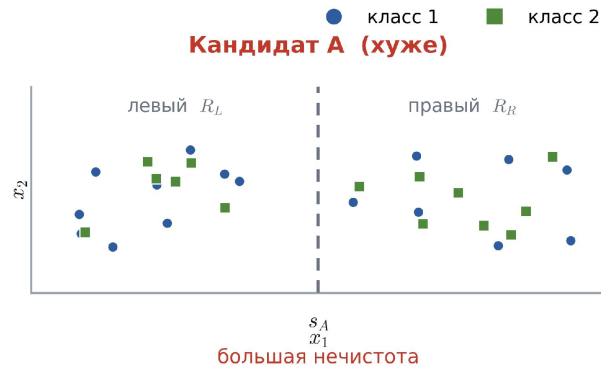
$$Q_{\text{split}}(j, s) = \frac{N_L}{N} Q(R_L(j, s)) + \frac{N_R}{N} Q(R_R(j, s)), \quad N = N_L + N_R$$

$Q(R)$ - мера нечистоты узла

Выбираем сплит с минимальной взвешенной нечистотой:

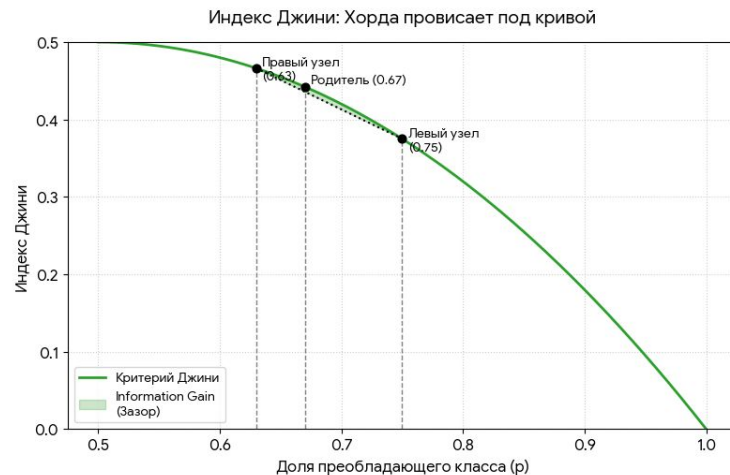
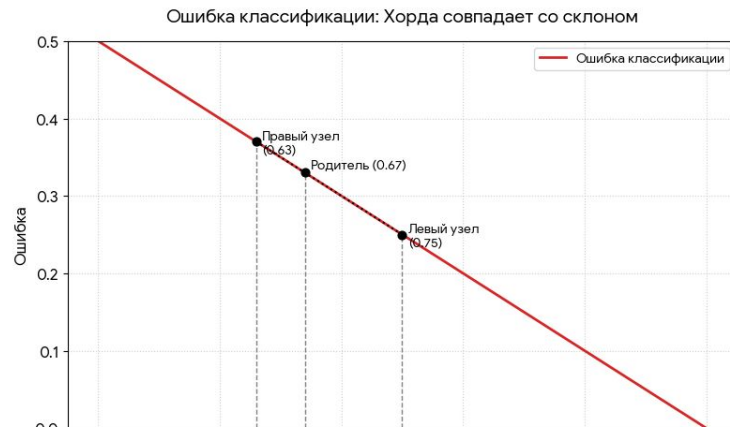
$$(j^*, s^*) = \arg \min_{j, s} Q_{\text{split}}(j, s)$$

Как посчитать однородность/“чистоту” в узле?



Меры нечистоты узла

● Индекс Джини (Gini index)	$Q_{\text{Gini}}(R_m) = 1 - \sum_{k=1}^K \hat{p}_{mk}^2$
● Энтропия (Entropy)	$Q_{\text{Ent}}(R_m) = - \sum_{k=1}^K \hat{p}_{mk} \log \hat{p}_{mk}$ ($0 \log 0 := 0$)
● Ошибка классификации (Misclassification error)	$Q_{\text{Mis}}(R_m) = 1 - \max_k \hat{p}_{mk}$



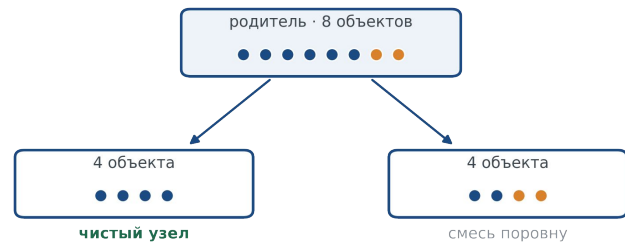
Ошибка классификации

1 Излом и постоянная производная

В точке $p = 0,5$ у функции излом, производной нет. На остальных участках производная постоянна.

2 Нулевой выигрыш от сплита

График линейен, поэтому взвешенная ошибка потомков в точности равна ошибке родителя: выигрыш от разбиения **равен нулю**, и сплит выбрать не из чего.



	родитель	потомки	выигрыш
Ошибка классификации	0,25	0,25	0
Индекс Джини	0,375	0,25	0,125

Родитель: 6 / 2 → L: 4 / 0 R: 2 / 2

Критерий	Родитель	Левый	Правый	Q_{split}	Выигрыш
Ошибка классификации	$1 - \frac{6}{8}$ = 0.25	$1 - \frac{4}{4}$ = 0	$1 - \frac{2}{4}$ = 0.5	$\frac{4}{8} \cdot 0 + \frac{4}{8} \cdot 0.5$ = 0.25	$0.25 - 0.25$ = 0
Индекс Джини	$1 - \left(\frac{6}{8}\right)^2 - \left(\frac{2}{8}\right)^2$ = 0.375	$1 - \left(\frac{4}{4}\right)^2$ = 0	$1 - \left(\frac{2}{4}\right)^2 - \left(\frac{2}{4}\right)^2$ = 0.5	$\frac{4}{8} \cdot 0 + \frac{4}{8} \cdot 0.5$ = 0.25	$0.375 - 0.25$ = 0.125

$$\text{Gain} = Q(\text{parent}) - Q_{\text{split}}$$

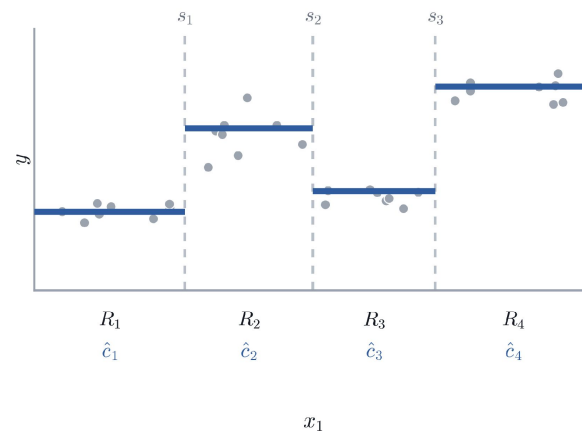
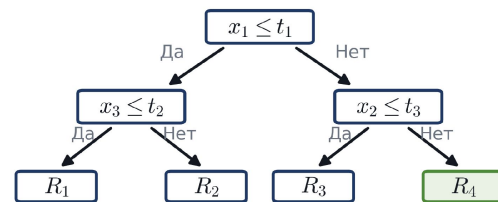
Задача регрессии

Идея: дерево разбивает пространство признаков на области и предсказывает в каждой области константу

$$\hat{f}(x) = \sum_{m=1}^M c_m I(x \in R_m).$$

$$x \in R_m \Rightarrow \hat{f}(x) = c_m.$$

То есть каждый лист R_m задает свое предсказание c_m (среднее, если минимизируем MSE и медиану - если MAE).



Критерий CART для регрессии

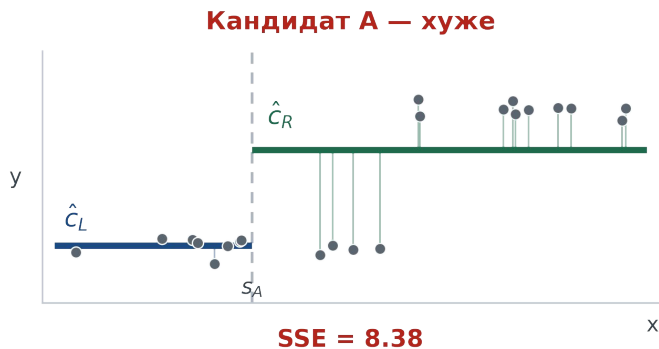
Для каждого кандидата (j, s) на предикат оцениваем, насколько хорошо левый и правый дочерние узлы аппроксимируют таргет y

$$\min_{j, s} \left[\sum_{x_i \in R_L(j, s)} (y_i - \hat{c}_L)^2 + \sum_{x_i \in R_R(j, s)} (y_i - \hat{c}_R)^2 \right].$$

$$\hat{c}_L = \text{ave}(y_i \mid x_i \in R_L(j, s)), \quad \hat{c}_R = \text{ave}(y_i \mid x_i \in R_R(j, s)).$$

CART строится **жадно**:

1. Выбираем кандидата на предикат (j, s)
2. Считаем средние $\hat{c}_L$ и $\hat{c}_R$
3. Выбираем split с минимальной суммой квадратов ошибок



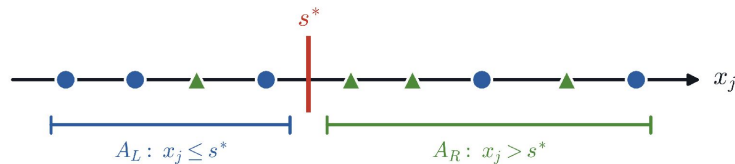
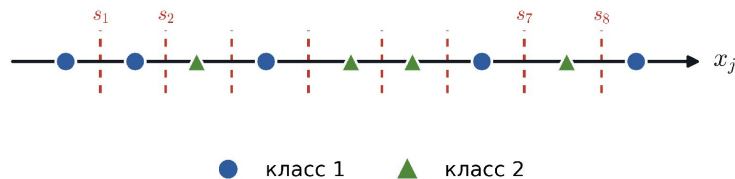
Часть 3. Как CART строит дерево

Поиск порога

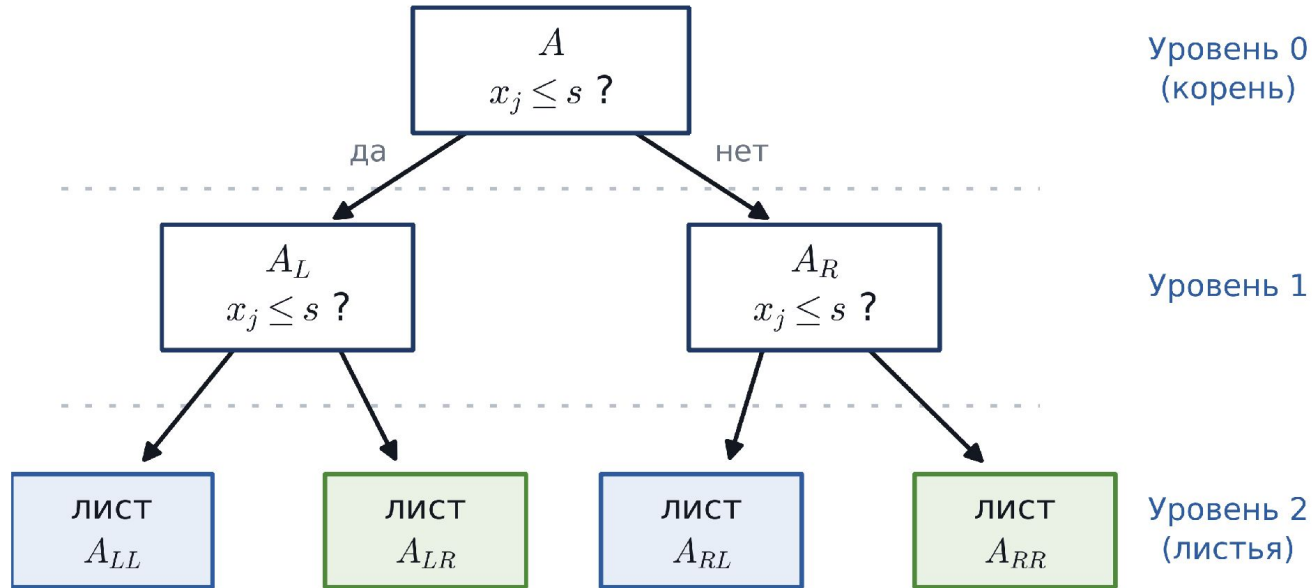
Идея: для каждого признака значения внутри текущего узла сортируются. Кандидатные пороги берутся между соседними различными значениями. Split делаем по тому порогу признака j , при котором критерий split-a минимален:

- для классификации - weighted impurity,
- для регрессии - сумма квадратов ошибок.

Значения признака в узле отсортированы; кандидаты — середины между соседними



Жадный рекурсивный CART



Рекурсия продолжается, пока не выполнится условие остановки

Про жадность

Перебор кандидатов в одном узле: признак, порог, критерий

```
best_score = +inf

for j in features:
    values = sorted(unique(x[i][j] for i in A))

    for r in range(len(values) - 1):
        s = (values[r] + values[r+1]) / 2

        A_L = {i in A : x[i][j] <= s}
        A_R = {i in A : x[i][j] > s}

        score = split_criterion(A_L, A_R)

    if score < best_score:
        best_score = score
        best_split = (j, s)
```

Сплит выбирается локально

В каждом узле берётся кандидат, лучший на данном шаге. Последствия для потомков не учитываются, назад алгоритм не возвращается.

Почему не глобально

Точное построение оптимального бинарного дерева решений – NP-полная задача.

Часть 4. Регуляризация дерева

Pre-pruning

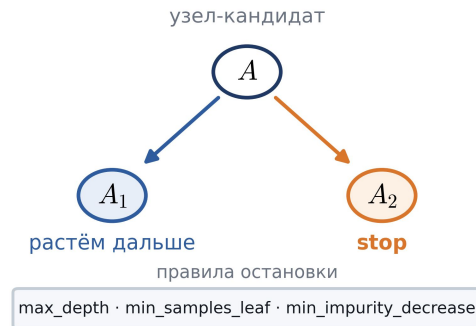
Идея: предварительная обрезка ограничивает рост дерева заранее, до построения полного дерева

Типичные правила останова:

- `max_depth`
- `min_samples_split`
- `min_samples_leaf`
- `min_impurity_decrease`
- узел уже достаточно чистый

Уменьшаем дисперсию, увеличиваем bias.

Pre-pruning вычислительно дешевый, но можем упустить полезное разбиение



Разрешим дальнейший сплит только, если:

$$\Delta Q(A, j, s) = Q(A) - \frac{|A_L|}{|A|} Q(A_L) - \frac{|A_R|}{|A|} Q(A_R) \geq \varepsilon$$

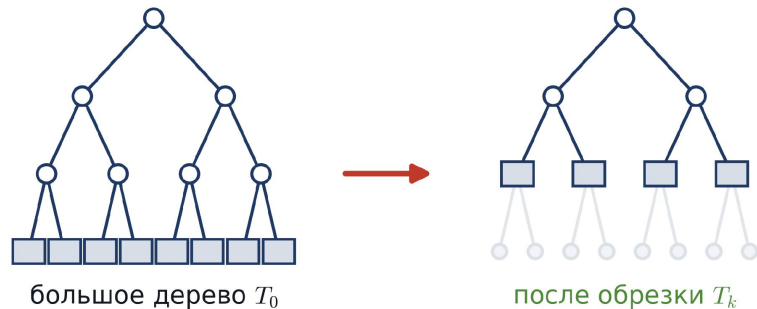
Апостериорная обрезка

Сначала строим большое дерево, которое хорошо подгоняет обучающие данные. Затем последовательно удаляем ветви, чтобы найти баланс между качеством подгонки и сложностью дерева.

$$C_{\alpha}(T) = \hat{R}_S(T) + \alpha |T|$$

$|T|$ - число терминальных узлов

Ветвь с минимальным $\alpha(t)$ обрезается первой: она дает наименьший выигрыш качества на один дополнительный лист (каждый дополнительный лист должен достаточно сильно уменьшать ошибку, чтобы оправдать свою “цену”)



Weakest-link pruning

$$\alpha(t) = \frac{\hat{R}_S(t) - \hat{R}_S(T_t)}{|T_t| - 1}$$

удаляем ветвь, для которой цена упрощения минимальна

1

grow large tree

→

2

compute pruning sequence

→

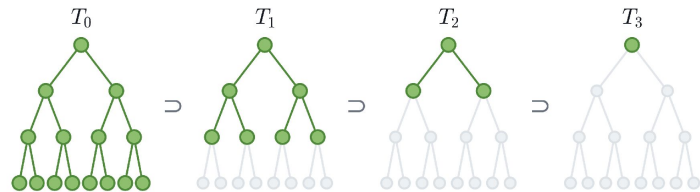
3

choose α by validation / CV

Pruning sequence

Алгоритм weakest-link pruning порождает вложенную последовательность деревьев

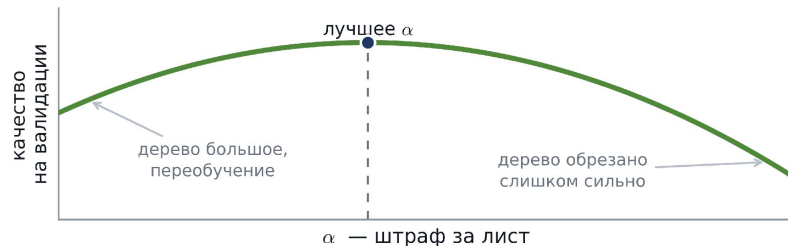
$$T_0 \supset T_1 \supset T_2 \supset \dots \supset T_K$$



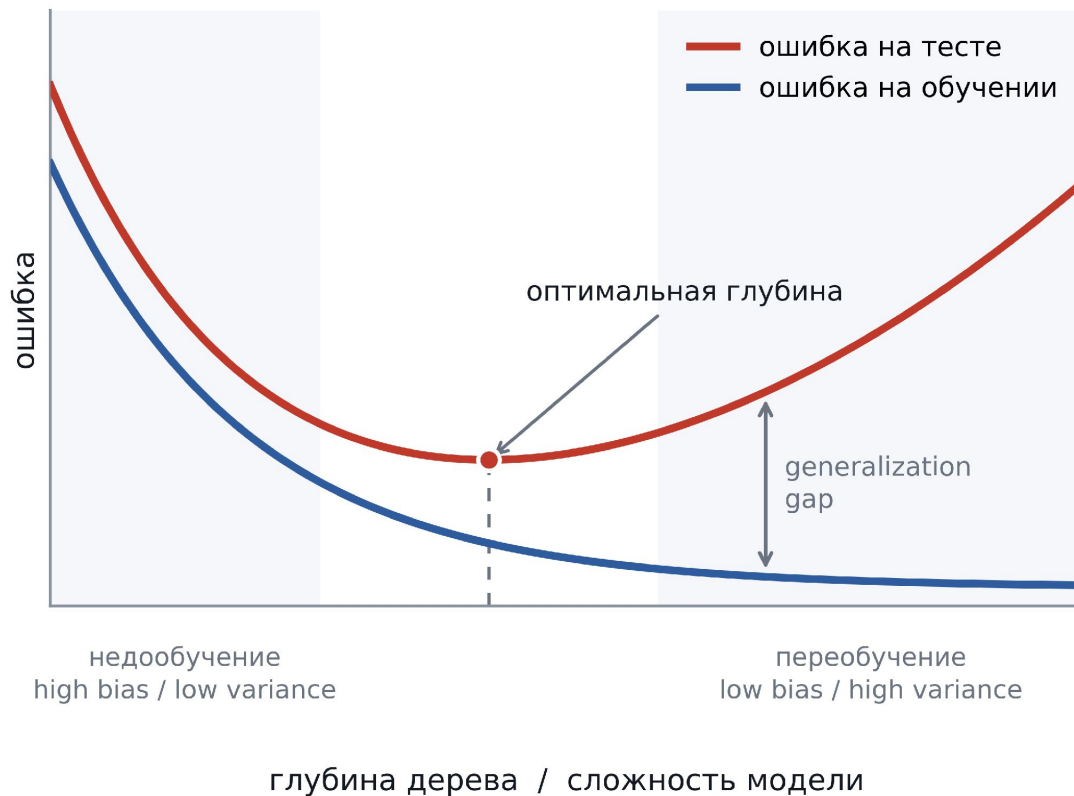
Каждый шаг удаляет ветвь с минимальным $\alpha(t)$ — деревья вложены друг в друга

Выбор модели

- выбираем α по валидации или кросс-валидации
- итоговое дерево — то, что даёт лучшее качество на отложенной выборке



Bias-variance перспектива на глубину дерева



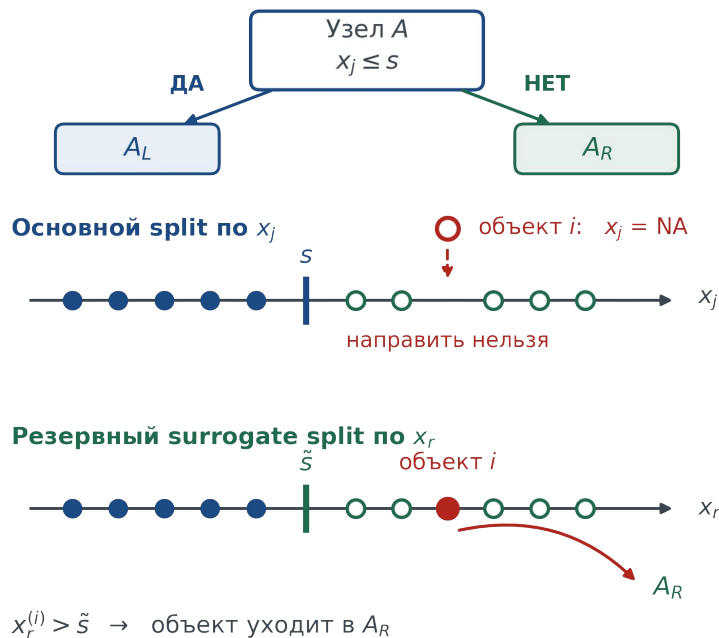
Часть 5. Ограничения CART. Мотивация ансамблей

Пропущенные значения, суррогатный сплит

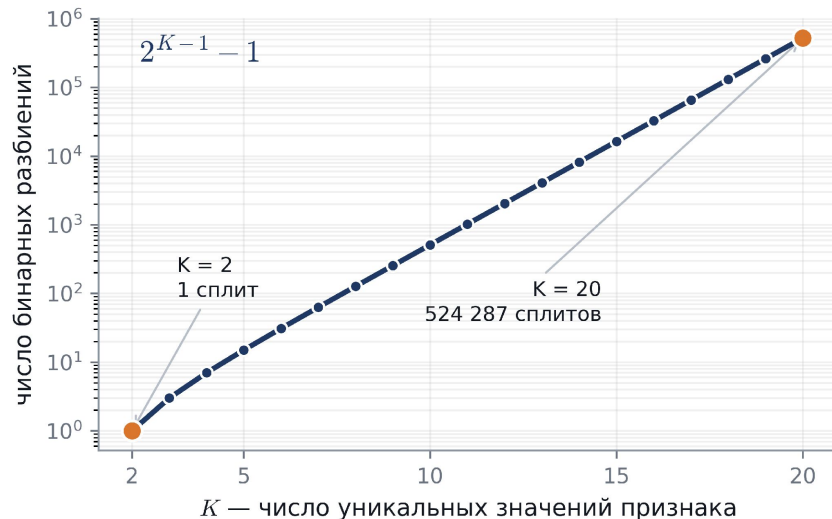
Проблема: если в узле выбран предикат $x_j \leq s$, то объект с пропуском в данном признаке нельзя направить в конкретный узел.

Решение: подберем суррогатный сплит, максимально хорошо воспроизводящий разбиение основного предиката

$$\text{Agree}(\tilde{j}, \tilde{s}; j, s) = \frac{1}{|A_{\text{obs}}|} \sum_{i \in A_{\text{obs}}} 1[1(x_{ij} \leq s) = 1(x_{i\tilde{j}} \leq \tilde{s})]$$



Высококардинальные переменные



Количество бинарных разбиений для K значений данного признака:

$$N_{\text{splits}} = 2^{K-1} - 1.$$

При большом K число кандидатов быстро растет, а вместе с ним и шанс найти случайный “удачный” сплит. Поэтому CART часто предпочитает ZIP code, user ID и подобные фичи.

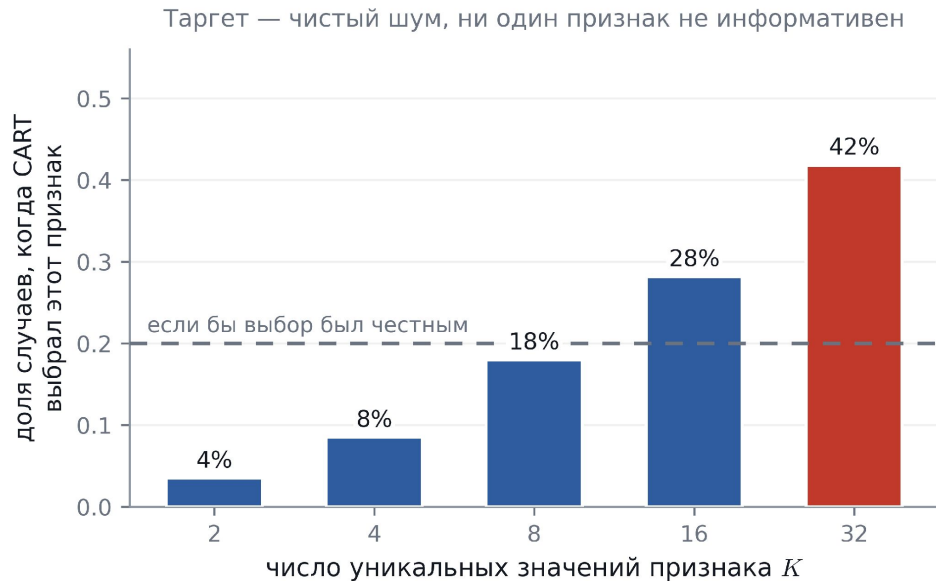
Но **высокая кардинальность $\neq$ высокая информативность признака**

Variable selection bias

До сих пор мы говорили: CART выбирает split с максимальным уменьшением нечистоты. Но это не всегда означает, что выбранный признак действительно наиболее информативен. Если у одного признака больше допустимых split-кандидатов, у него также больше способов случайно хорошо разделить train (уменьшить нечистоту в дочерних узлах)

Bias возникает не в формуле Q , а в

exhaustive search по неодинаковым множествам кандидатов.



Conditional inference деревья (условного вывода)

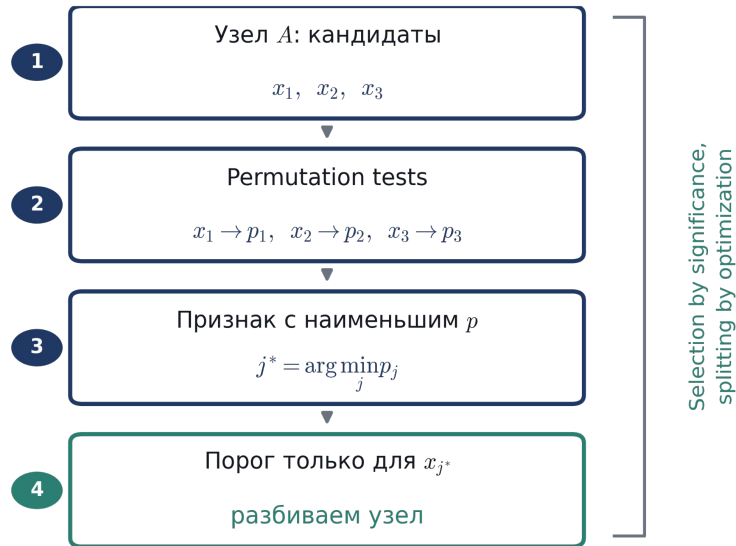
Идея: давайте сначала проверять, связан ли вообще данный признак с таргетом

1. Для каждого признака x_j в узле A тестируем $H_{0,j}: x_j \perp\!\!\!\perp y$ (условная независимость) с помощью permutation test

$$j^* = \arg \min_j p_j$$

Если $\min_j p_j > \alpha_{\text{test}}$, разбиение не выполняем
и узел становится листом

2. Только после выбора переменной j^* ищем лучший сплит внутри неё



Permutation test

Идея

сравниваем наблюдаемую связь с тем, что могло бы возникнуть случайно.

1

Считаем статистику на исходных метках

разница средних x между двумя классами

2

Перемешиваем только метки y

значения x остаются на своих местах

3

Повторяем и сравниваем

если наблюдаемая связь реальна, перестановки её разрушают

Статистика T - это мера связи. Перестановки показывают, какие её значения бывают, когда связи нет.

Исходные метки

x	1	2	3	4	5	6	7	8
y	0	0	0	0	1	1	1	1

x при $y = 1$: 5, 6, 7, 8 $\rightarrow \bar{x} = 6.5$

x при $y = 0$: 1, 2, 3, 4 $\rightarrow \bar{x} = 2.5$

$$T^{\text{obs}} = |\bar{x}_{y=1} - \bar{x}_{y=0}| = |6.5 - 2.5| = 4$$

наблюдаемая разница средних

↓ перемешиваем только y

Перестановка №1

x	1	2	3	4	5	6	7	8
y	0	1	0	1	1	0	1	0

x при $y = 1$: 2, 4, 5, 7 $\rightarrow \bar{x} = 4.5$

x при $y = 0$: 1, 3, 6, 8 $\rightarrow \bar{x} = 4.5$

$$T^{(1)} = |4.5 - 4.5| = 0$$

связь исчезла — значит, она держалась на метках

Permutation test: от перестановок к p-value

p-value

доля перестановок, на которых статистика оказалась не меньше наблюдаемой:

$$\hat{p}_j = \frac{1 + \sum_{m=1}^M \mathbb{1}\{T_j^{(m)} \geq T_j^{\text{obs}}\}}{M + 1}$$

$$N_{\text{perm}} = \binom{8}{4} = 70, \quad p_j = \frac{2}{70} = 0.0286 < \alpha_{\text{test}} = 0.05$$

Что это значит

- $p < 0.05$ - независимость x и y отвергаем
- признак попадает в кандидаты на split
- и только потом ищем лучший порог внутри него

Единица в числителе и знаменателе - сама наблюдаемая расстановка меток.



Наблюдаемое значение попало в самый хвост: случайные перестановки почти никогда не дают такой сильной связи.

Основные ограничения дерева решений

№	Ограничение	Что это означает
1	Высокая variance	малые изменения данных радикально меняют дерево
2	Жадное построение	нет гарантии глобально оптимальной структуры
3	Axis-aligned splits	границы ступенчатые, ограниченно гибкие
4	Переобучение	глубокое дерево подгоняет train, но плохо обобщает
5	Selection bias	предпочтение признакам с большим числом кандидатов
6	Слабая экстраполяция	вне обученного диапазона тренд не продолжается

Переходим к ансамблям

Метод	Главная идея	Что исправляет относительно CART
Random Forest	bagging + случайный выбор подмножества признаков в каждом узле	снижает variance, уменьшает корреляцию между деревьями
ExtraTrees	ещё больше рандомизации: случайный признак и/или случайный порог	дополнительно декоррелирует деревья, часто ускоряет обучение
GBDT	деревья добавляются последовательно, каждое исправляет ошибки предыдущих	снижает bias и даёт сильное итоговое качество

Необходимость в ансамблях

CART остаётся слабой базовой моделью, а ансамбли **снимают его практические ограничения.**

Random Forest

variance ↓

ExtraTrees

variance ↓↓

GBDT

bias ↓

ИСТОЧНИКИ

1. Breiman, L., Friedman, J. H., Olshen, R. A., Stone, C. J. (1984). *Classification and Regression Trees*. Wadsworth.
2. Hastie, T., Tibshirani, R., Friedman, J. (2009). *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*. 2nd ed. Springer.
3. Deisenroth, M. P., Faisal, A. A., Ong, C. S. (2020). *Mathematics for Machine Learning*. Cambridge University Press.
4. Bruce, P., Bruce, A., Gedeck, P. (2020). *Practical Statistics for Data Scientists*. 2nd ed. O'Reilly.
5. Huyen, C. (2022). *Designing Machine Learning Systems: An Iterative Process for Production-Ready Applications*. O'Reilly.
6. Hyafil, L., Rivest, R. L. (1976). Constructing optimal binary decision trees is NP-complete. *Information Processing Letters*, 5(1), 15–17.
7. Bertsimas, D., Dunn, J. (2017). Optimal classification trees. *Machine Learning*, 106, 1039–1082.
8. Hothorn, T., Hornik, K., Zeileis, A. (2006). Unbiased recursive partitioning: A conditional inference framework. *Journal of Computational and Graphical Statistics*, 15(3), 651–674.
9. Strobl, C., Boulesteix, A.-L., Zeileis, A., Hothorn, T. (2007). Bias in random forest variable importance measures: Illustrations, sources and a solution. *BMC Bioinformatics*, 8, 25.
10. Scikit-learn developers. *Decision Trees; Minimal Cost-Complexity Pruning*. Scikit-learn documentation.
11. Breiman, L. (2001). Random forests. *Machine Learning*, 45, 5–32.
12. Geurts, P., Ernst, D., Wehenkel, L. (2006). Extremely randomized trees. *Machine Learning*, 63, 3–42.
13. Friedman, J. H. (2001). Greedy function approximation: A gradient boosting machine. *The Annals of Statistics*, 29(5), 1189–1232.